# Software Architecture Report

**Name: Celestia Shavgulidze**

**Course: Cybersecurity with Software Development**

**Subject: Software Architecture**

**Teacher's name: Naveena**

**Submission Date: 13/12/2025**

**Table of contents**


| |
|---|
| **Part1** |
| **Part2** |
| **Part3** |
| **Part4** |


# Part 1

### a. Software architecture:

Software Architecture is a high-level structure of a software system that defines how different components of systems interact, how data flows between them, and what design patterns or technologies are used to find and build the solution. It is the blueprint of a system, like architectural plans of a building, outlining major components, their responsibilities, and the relationships between them before the thorough code development begins.

#### Importance in software design:

Software architecture is fundamental and critical to software design as it firstly provides clarity and structure during the development process. Without a proper architecture, systems become difficult to maintain, test, and extend, creating unnecessary issues that could have been easily avoided. Secondly, architecture ensures that the system is meeting both functional and non-functional requirements (what is required from software and performance, scalability, security). Good architecture design prevents risks in early development by identifying technical challenges in advance, selecting more suitable frameworks, and better integration plans. Thirdly. It helps provide clearer communication as architecture documents help developers, project managers, testers, and many other team members understand the system at a higher level and enables better coordination and decision-making.

### b. Main goals and purposes of software development methodologies:

Software development methodologies provide structured approaches and instructions for planning, developing, testing, and delivering projects. Its main goals include:

1. **Improved project organisation** – Methodologies define steps, roles and responsibilities, helping teams work in a more disciplined and predictive manner.
2. **Risk reduction** – Methodologies allow potential issues to be spotted and prevented earlier (technical, financial, time-line related, etc) by planning and reviewing work at different stages.
3. **Higher product quality** – Methodologies integrate testing, documentation, and review processes, resulting in software that meets industry standards and user requirements.
4. **Better communication and coordination** – Structured processes ensure that all stakeholders (developers, clients, tester, managers) share same expectations and understanding of the projects.

Agile methodologies allow and quide teams to develop software fully and ensures that the final product matches the requirements, is maintainable, and reliable.

### C. Agile methodology:

Agile methodology is an iterative approach to software development, emphasising on flexibility, customer collaboration, and continuous improvement. Instead of planning everything in the beginning at once, Agile teams break projects into smaller tasks/cycles, called sprints or iterations where teams plan, develop, test, and review parts of software as the project progresses. Thanks to the Agile methodology, projects rapidly adapt to changes and early delivery of the working solutions.

#### Two Key Principles of Agile

1. Customer collaboration over contract negotiations

Agile methodology promotes continuous interaction with customers throughout the project. Instead of deciding on all the requirements upfront, Agile teams regularly gather feedback and make changes based on evolved needs.

Example: Scrum

Scrum is one of the most popular Agile methodologies in the industry. It's based on iterative development where the workload is broken down in smaller tasks and is completed in shot, fixed-length cycles called "Sprints," usually lasting two to four weeks. Each sprint's result gives a potentially shippable product increment.

Scrum defines specific roles, artefacts, and responsibilities to ensure structure and accountability while remaining flexible.

Key roles in scrum are:

- **Product Owner** – Product Owner is responsible for defining and
- **Scrum Master** – Scrum Master facilitates the Scrum process, removes obstacles, and ensures Agile principles are followed.
- **Development Team** – The development team is a cross-functional group that designs, develops, and tests the product.

Each scrum event provides regular planning and feedback.

- **Sprint Planning** – During this stage, the team will select backlog items to complete during the sprint.
- **Daily Scrum** – Daily scrum is a short daily meeting to track progress and identify issues.
- **Sprint Review** – Customer reviews the complete work and provide feedback.
- **Sprint Retrospective** – The team reflects on the sprint and identifies process improvements.

Scrum artefacts include the product backlog, sprint backlog, and product increment, ensuring transparency and progress tracking. It's considered Agile because it supports continuous improvement, customer feedback, frequent delivery of working software, and gives the ability to respond quickly to changes.

Example: If a customer's requirements change after a sprint review, new or updated features can be prioritised for the next sprint without disrupting the rest of the project.

In industry implementations of Scrum, daily Scrum meetings are conducted as short stand-up meetings, typically lasting around 15 minutes.

2. Response to changes over following a fixed plan

Traditional methods, such as Waterfall, follow a strict and sequential plan while Agile recognises that requirements change and evolve, be it because of new business goals or customer feedback.

Example: Kanban

Kanban uses visual boards with columns (or "sticky notes") such as, "To Do," "In Progress," and "Done."

If plans change, items can be easily reordered or added without interrupting the development flow, reducing bottlenecks and supporting continuous delivery.

**d. The advantages (2) and disadvantages (2) of Agile Development methodologies**

**Advantages:**

**1. Better flexibility and adaptability**

Agile allows changes at any stage of the project without interrupting the workflow and making it easier to make changes in environments where requirements are uncertain or evolve over time. Waterfall, in contrast, locks requirements easily into the project, making late changes expensive and more difficult to squeeze in.

**2. More efficient delivery of working software**

Agile allows to deliver small, functional pieces of the project frequently. This lets customers see progress early and provide feedback. With Waterfall, the final product is only delivered at the end of the development cycle.

**Disadvantages:**

**1. Less predictable timelines and expenses**

Since Agile allows changes throughout the project, predicting the final budget or delivery schedule can be difficult. Waterfall typically provides more predictable planning, as everything is agreed upon upfront.

**2. Requirements for high team collaboration and discipline**

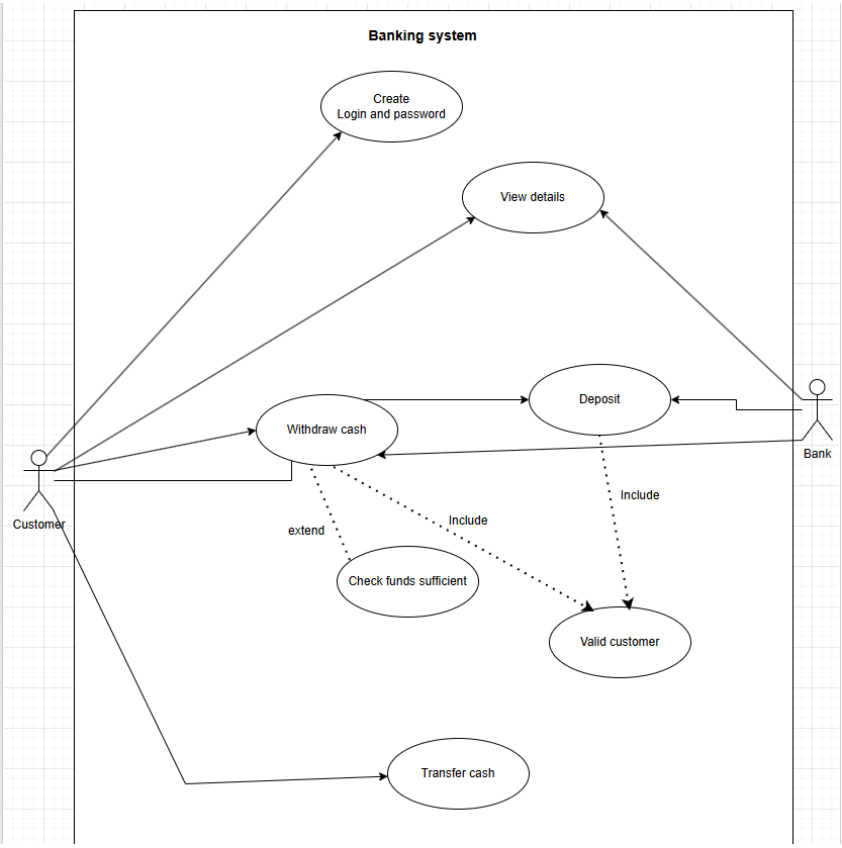
Agile heavily relies on communication, daily meetings, and ongoing involvement of the customer and the team. If the team is unexperienced or the customer is unavailable, this can make the project more difficult to complete. Since Waterfall is more structured, it can make the job easier for unexperienced teams to complete tasks.

## Part -2

1. **The purpose of use-case diagram:**

The purpose of a use case diagram is to provide a high-level visual representation of the functional requirements of a system. it identifies the interactions between “actors” (users or external systems) and the specific “use cases” (actions or services) the system provides. This helps stakeholders understand what the system does without getting bogged down in technical code.

2. **Use case diagram**



## Part 3

1. class diagrams for the banking application

| Class name | | |
|---|---|---|
| - Attributes | | |
| Profile | Bank | Application |
| - username: String<br>- password: String<br>- balance: double | - users: ArrayList | |
| + methods | | |
| Profile | Bank | Application |
| + deposit(amount: double)<br>+ withdraw(amount: double) | + addUser(u: Profile)<br>+ getUser(user: String)<br>+ login(user: String, pass: String) | + main(args: String[])<br>+ userMenu(u: Profile) |

## Part 4

a. Explain the importance and benefits of testing a system

Software testing is critical because it ensures the reliability, security, and performance of a system before it reaches the end user. The primary benefits include risk mitigation (finding bugs before they cause financial loss), quality assurance (ensuring the system meets the requirements in the Use Case diagram), and user trust. For a banking system, testing is especially vital to ensure that transactions are accurate and that user credentials remain secure.

b. test case.

A test case is a specific set of conditions, inputs, and expected variables used by a tester to determine if a specific feature of a software system is working correctly. It acts as a 'pass/fail' check for a single unit of logic.

a. Derive a test case for the above use case diagram, including the following elements.

- ID : TC-001
- Name : Withdraw Cash Transaction
- Actor : Customer
- Description : To verify that the banking system allows a logged-in customer to withdraw a valid amount of cash and correctly updates the account balance.

- Preconditions :
  1. The customer has created a profile.
  2. The customer is logged into the system.
  3. The account balance is greater than the withdrawal amount (e.g., Balance: €100).
- Expected Result :
  1. The system processes the withdrawal.
  2. The current balance is reduced by the withdrawal amount.
  3. A "Withdrawal Successful" confirmation is displayed.
- Actual Result : As expected. The balance was updated from €100 to €50 after a €50 withdrawal, and the success message was shown.

```
Welcome to the Banking System

1. Create Profile
2. Login
3. Exit
1
Enter username: Cele
Enter password: pass1234
Initial deposit: 100
Profile created successfully!

1. Create Profile
2. Login
3. Exit
2
Username: Cele
Password: pass1234

--- User Menu ---
1. Check Balance
2. Deposit
3. Withdraw
4. Logout
3
Amount: 50
Withdrawal successful. New balance: €50.0

--- User Menu ---
1. Check Balance
2. Deposit
3. Withdraw
4. Logout
4

1. Create Profile
2. Login
3. Exit
```

## Does the system work?

Yes, the banking system is operational and performs all the required operations smoothly. As demonstrated in the testing phase screenshot, the system allows the creation of new user profiles, handles secure login authentication, and correctly processes financial transactions.

## If not, what problems did you have?

During the initial development in Eclipse, I encountered a few "module-info" errors and package configuration issues that prevented the classes from communicating. However, by removing the module-info file and using the default package structure, I was able to resolve these technical glitches and get the three classes (Profile, Bank, and Application) to work together seamlessly.

## What did you find difficult or tricky with the assignment?

The trickiest part of the assignment was transitioning the architectural design from the Use Case and Class diagrams to the actual Java code.

## Reflecting on how testing results align with expectations and suggesting future improvements.

The testing results aligned perfectly with my initial expectations. The derived test case (TC-001) successfully proved that the system handles valid withdrawals and updates the balance accurately, which was the primary goal of the system architecture.

For future improvements, the system could be enhanced by adding a Data Persistence layer (such as a SQL database) so that user profiles are saved permanently instead of being wiped when the program closes.

Additionally, implementing a Graphical User Interface (GUI) using JavaFX would make the system more user-friendly than the current console-based interface.